Paso 6 — Configuración del build (y el build mismo): qué se genera, con qué ajustes, qué se queda y cuántas se probaron

Fecha: 2026-10-08 Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.

Para: el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 6 de `docs/AgentPDFs/WORKFLOW.md`: la doctrina de construcción (`assets/_build.yaml` → `sqx/projects/doctrine.py`), los filtros de aceptación (`assets/_study.yaml` → `core/buildfilters.py` → `sqx/projects/rankings.py`), la paleta de bloques (`sqx/blocks/`, `sqx/projects/buildingblocks.py`, agente `buildingBlocksExpert`), la búsqueda genética y su métrica, la regla dura 15, y la tarea Build de SQX corriendo.

Qué es: el documento de consulta del paso 6, con la misma forma que los de los pasos 8 a 19. Este paso no juzga con estadística: configura una tarea de SQX y la deja correr. Por eso la sección 2 explica mecánica (qué es un build, qué es una paleta) y una sola idea estadística, que es la que manda sobre todo lo que viene después: **cuántas estrategias se probaron para quedarse con las que quedan**.

De dónde salen los números. Todos los ejemplos son de la corrida `Test_USDJPY_M30_doubleROCLong_10k_B5` del 2026-10-08 (USDJPY, M30, largas, plantilla `doubleROCLong`, paleta `doubleROCLong_USDJPY_M30`), la más reciente que construyó algo: el build corrió de 01:25:47 a 02:55:51. Están leídos por mí de tres sitios: el `project.cfx` archivado (`AlgoData/projects/retired/SQX_w2/Test_USDJPY_M30_doubleROCLong_10k_B5-2026-10-08.tar.gz`), el databank «Results» exportado (`AlgoData/metrics/…/Results/metrics.csv`, `AlgoData/raw/…/Results/2026-10-08/trades.parquet`, `AlgoData/harvest/…/metrics.parquet`) y la fila del paso 6 en el ledger (`AlgoData/ledger/USDJPY_M30_doubleROCLong.jsonl`). Lo que es cuenta mía y no salida de ningún estudio lleva la marca **«calculado por mí»**; los scripts están en la carpeta temporal `…/scratchpad/paso06/` (`databank.py`, `structure.py`, `trials.py`). «Ilustrativo» quiere decir cuenta hecha a mano para enseñar, no medida.

El commit que describo: `b2b4c8c`. La corrida se hizo con `7203f4c-dirty`; entre los dos commits no cambió ningún fichero del paso (`git diff 7203f4c b2b4c8c` vacío sobre `assets/`, `sqx/projects/`, `sqx/blocks/`, `core/buildfilters.py`, `bin/workflow.sh`). **Ficheros del paso con cambios sin confirmar** (todos con fecha anterior a la corrida, así que la corrida los usó tal como están hoy): `assets/_build.yaml`, `assets/_study.yaml`, `sqx/blocks/taxonomy.yaml`, `sqx/projects/buildrules.py`, `sqx/projects/tasksettings.py`, `bin/workflow.sh`, `ledger/steprows.py`, `knowhow/research/ga-target-choice.md`; y sin seguimiento de git: `sqx/projects/buildcensus.py`, `studies/screening/batches/` (el `report.py` que cito) y nueve paletas, entre ellas la de la corrida y las seis `_v3`. Los números de línea de `bin/workflow.sh` (:51, :59, :67) valen para el árbol de trabajo actual, no para `HEAD`.

Ninguna línea de este documento está congelada. `assets/_study.yaml` lo dice en su tercera línea: «PROPUESTA DE AGENTE, PROVISIONAL hasta que el dueño la apruebe».

---

0. En una página

La pregunta del paso. ¿Qué forma pueden tener las estrategias que SQX genera, con qué piezas, cuánto tiempo busca, y cuáles de las generadas se guardan?

Lo que hay cargado hoy:

- **Forma** (`_build.yaml`): una dirección, sólo órdenes a mercado, salida obligatoria por barras entre 2 y 24 horas, sin stop ni objetivo, como mucho 2 condiciones por lado, sólo la última barra cerrada.
- **Búsqueda:** genética, 20 islas de 250, 25 generaciones, cruce 89 %, mutación 20 %; optimiza el **Ret/DD del tramo `build`**; para por tiempo.
- **Aceptación** (`_study.yaml`, PROVISIONAL): entre 10 y 75 operaciones al año, profit factor ≥ 1,0 y beneficio neto > 0, todo en el `build`.
- **Paleta:** una lista corta de condiciones por familia de idea; nunca bloques de reloj ni niveles de órdenes stop/limit.
- **Criterio del piloto:** ninguno. `pipeline/autopilot/criteria.yaml` no tiene paso 6: el paso no juzga.

Lo que salió en la corrida real (90 minutos de build):

	estrategias	sobre las generadas
generadas y probadas por SQX	**832.299**	100 %
aceptadas (cumplieron los filtros)	47.976	5,76 %
guardadas en el databank «Results»	10.000	1,20 %
vivas tras el paso 8 (dato de la corrida)	72	no se mide contra las generadas: sólo las 10.000 guardadas se probaron en `oos1`, así que la tasa es 72 de 10.000 = 0,72 %

Referencia del azar (calculado por el revisor a partir de `falsePositives/funnel.csv`, control de monos del primer lote de esta celda, 2026-10-07; método: 33 de 5.000 estrategias de entrada aleatoria, sin ventaja, cruzan el paso 8 con el mismo `criteria.yaml`, y se escala a 10.000; intervalo de Clopper-Pearson, sin corregir que los 5.000 monos salen de sólo 100 donantes): 33 vivas (2 Pasa y 31 Limbo) = 0,66 %. Sobre 10.000 serían unas **66 vivas por puro azar** (entre 45 y 93). Los cinco lotes reales dieron 56, 48, 10, 31 y 72. Las 72 de esta corrida caben dentro de lo que da el azar con estas reglas. Límites de la comparación: los monos no pasaron por la selección del build, se midieron en otro lote de la celda y `criteria.yaml` es provisional.

Las nueve cosas que conviene saber antes de leer el resto:

1. **El número de ensayos del build no es 10.000, es 832.299, y las dos cifras miden cosas distintas.** SQX lo escribe en su log al acabar la tarea y `ledger/steprows.py:40` lo apunta en el ledger. 832.299 mide cuánto se buscó en el `build`, o sea cuánto está inflado lo que el `build` enseña (la mejor de 832.299 sin ventaja tendría t ≈ 4,8 en el `build`); 10.000 es cuántas estrategias se prueban en `oos1`, porque sólo esas se retestean. El resumen del piloto dice «CONSTRUCCION: 0 → 10000» y el informe de lotes llama «generadas» a las 10.000 guardadas (`studies/screening/batches/report.py:43` y `:53`). **Ninguna regla de los pasos 8 a 25 lee hoy ninguna de las dos N:** las reglas del paso 8 comparan cada estrategia con umbrales fijos (t ≥ 1,65), sin corrección por cuántas se juzgan. La N la leen, además de `ledger/report.py:92,103`, la ventana (`ui/daemon/ops/ledgerview.py:55`), el archivo (`core/archive/manifest.py:76`) y el veredicto oos2 de la cartera (`portfolio/common/construct/oos2.py:27,69`), que deflacta el Sharpe con la N total de todos los estudios; esa N ya incluye las filas «generated» del paso 6 (`ledger/trials.py:43`), 4.880.561 sólo en esta celda.

2. **Los filtros escritos casi no deciden nada; decide el databank lleno.** Con el databank a tope, una estrategia nueva sólo entra si mejora a la peor en Ret/DD. En la corrida, la de peor Ret/DD (2,90) tiene profit factor 1,19 y neto 50.890 USD (el profit factor mínimo del databank es 1,14, otra estrategia con Ret/DD 3,48; el neto mínimo, 12.702): el filtro «PF ≥ 1,0 y neto > 0» quedó muy por debajo. El corte real es «las 10.000 de mejor Ret/DD del `build` entre 47.976». Las cifras de esta frase son «calculadas por el revisor» sobre `metrics.csv` (fila con Ret/DD mínimo y mínimos de columna).
3. **La métrica que optimiza (Ret/DD) no la escribe ningún módulo: viene del proyecto donante.** Y la ficha `knowhow/research/ga-target-choice.md` dice que la elección está sin resolver: el ruido entre dos corridas iguales (16 puntos) es mayor que la diferencia entre métricas.
4. **Con plantilla y sin hueco de salida, nunca hay condición de salida.** La doctrina la declara «optativa» y la tarea lleva `maxExitConditions="2"` y la salida por regla al 50 %, pero de las 3.496.906 operaciones de la corrida ninguna cerró por una regla: 87 % por barras y 13 % por el cierre del viernes.
5. **Las 10.000 son muy pocas ideas distintas.** Con esta plantilla y esta paleta sólo existen 104 formas posibles de regla (el hueco vacío, o uno de 103 bloques). El estudio cuenta 259 «estructuras» y cinco de ellas reúnen la mitad del databank. Lo demás son los mismos bloques con otros números.
6. **El 61 % del databank opera menos de 35 veces al año** (6.088 de 10.000; calculado por mí). El suelo cargado en el build es 10 al año. El paso 8 no aplica 35 como suelo: con menos de 35 deja en Limbo y sólo tira por debajo de 20 (`criteria.yaml:79-80`). En esta corrida, 57 de las 72 vivas operan entre 20 y 35 veces al año (calculado por el revisor sobre el `scorecard.parquet` del gate de B5); el paso 8 tira por esa regla unas 3.900 de 10.000, las de menos de 200 operaciones en 10 años (calculado por el revisor).
7. **Las cifras que justifican `_study.yaml` salieron de la calibración que mandaste borrar el 2026-10-04** (spread mal). El fichero las sigue citando en sus comentarios. Los umbrales están cargados, pero sin medida vigente detrás.
8. **El build corre un crosscheck que no filtra nada.** `MonteCarloManipulation` está encendido con sus dos condiciones apagadas: 30 simulaciones por estrategia que sólo gastan CPU. El comentario de `_build.yaml` que lo acompaña habla de otro crosscheck (el de alta precisión), que está apagado, y hoy el build no hace ninguna comprobación de robustez: filtra por número de operaciones, neto positivo y Ret/DD del `build` , y nada más.
9. **Las «72 vivas» no superan a las que deja pasar el azar.** Una población de entrada aleatoria, sin ventaja, medida con el mismo `criteria.yaml` en el primer lote de esta celda, deja el 0,66 % en el paso 8: unas 66 de 10.000 (45 a 93; calculado por el revisor, ver la referencia bajo la tabla). Los cinco lotes reales dieron 56, 48, 10, 31 y 72. Además, entre las estrategias del databank el nivel de t del `build` no se conserva en `oos1` (mediana 2,61 en el `build` , 0,74 en `oos1` ; calculado por el revisor).

---

1. Qué es el paso 6 y cómo decide

1.1 Dónde está

El paso 5 crea el proyecto (`python3 -m sqx.projects.builder`), y al crearlo ya escribe en la tarea Build la doctrina y los filtros. El paso 6 es esa configuración más la paleta, y la tarea corriendo. El paso 7 (retest en `oos1`) va encadenado en la misma sesión de SQX: el piloto los llama «6+7».

Con el comando único (`bin/workflow.sh` , regla dura 19) el orden real es:

1. `core.assets <SÍMBOLO>` (regla 5). Si sale con error, no se crea nada.
2. `sqx.projects.builder … --workflow --max-strategies N --minutes M` (`bin/workflow.sh:59`): clona el donante, escribe doctrina y filtros.
3. `sqx.projects.buildingblocks --palette …` (`bin/workflow.sh:67`): escribe la paleta en la tarea Build.
4. `pipeline.autopilot.run` : arranca SQX, la tarea Build corre hasta su tope de tiempo, sigue el retest OOS, se exporta.

1.2 Quién escribe cada cosa en la tarea Build

qué	de dónde sale	quién lo escribe
timeframe, motor, sesión, tamaño de posición, cierre del viernes, precisión, crosschecks	`assets/_build.yaml` y la ficha del activo	`doctrine.apply_doctrine` (`sqx/projects/doctrine.py:11`), en **todas** las tareas del proyecto
órdenes, tipos de salida, nº de condiciones, sin SL/PT	`assets/_build.yaml`	`sqx/projects/buildrules.py` , sólo en la tarea que genera
ajustes del genético	`sqx/projects/buildmode_model.xml`	`tasksettings.set_genetic` (`:164`), copia literal
genética o aleatoria	argumento `--random` del builder	`patch.set_generation`
tope de estrategias y de minutos	argumentos, o `databank.caps` por prefijo	`patch.set_caps` , `doctrine.caps` (`:155`)
filtros de aceptación	`assets/_study.yaml`	`rankings.apply` (`sqx/projects/rankings.py:45`)
bloques que se pueden sortear	una paleta de `sqx/blocks/palettes/`	`buildingblocks.apply` (`:74`)
métrica que optimiza	**el donante** (`XAUUSD_base_2026-09-21`)	**nadie**: se hereda
descartar las demasiado parecidas, descartes automáticos	el donante	nadie: se hereda
rango de periodos de los indicadores (2 a 200)	el donante	nadie: se hereda

1.3 Cómo «decide»

No hay veredicto Pasa / Limbo / No pasa. Hay tres mecanismos, y ninguno está en Python:

1. **Negativas del constructor** antes de crear el proyecto (apartado 4.1): si algo no está decidido, no se construye.
2. **Los filtros de aceptación** dentro de SQX: la estrategia que no los cumple no entra en el databank.
3. **El reemplazo por la métrica** cuando el databank está lleno: entra sólo la que mejora a la peor.

1.4 Tres avisos

1. **La corrida del ejemplo es una prueba (`Test_`), el quinto lote de un estudio de lotes** de la noche del 7 al 8 de octubre. Se lanzó con 10.000 estrategias y 90 minutos por argumento, no con los topes de doctrina de un `Test_` (300 y 10 minutos).
2. **El piloto se canceló a mano en el paso 8** («cancelado desde la ventana»). El build y el retest terminaron bien; lo que falta de la corrida es posterior a este paso.

3. **El log propio del proyecto ya no existe** (el proyecto se retiró). La cifra de generadas y aceptadas la tomo de la fila del ledger, que la leyó de ese log en su momento.

---

2. Fundamentos de este paso

2.1 Qué es un build en SQX

Un build es una máquina que **inventa estrategias, las prueba sobre un tramo de histórico y guarda las que cumplen unas condiciones**. Cada estrategia inventada pasa por un backtest completo sobre el tramo `build` (en USDJPY, del 2008-01-01 al 2017-12-31). Hay dos formas de inventar:

- **Generación aleatoria.** Cada estrategia se sortea desde cero: se eligen bloques al azar de la lista permitida y números al azar para sus parámetros. Ninguna estrategia sabe nada de las anteriores.
- **Evolución genética.** Se sortea una primera tanda (la población inicial) y, a partir de ahí, las siguientes se fabrican **a partir de las mejores de la tanda anterior**.

2.2 La evolución genética, con un ejemplo de cuatro estrategias

Ilustrativo. Supón una población de cuatro estrategias y que la métrica es el Ret/DD:

	regla de entrada	salida	Ret/DD
A	RSI(14) cruza 30 hacia arriba	10 barras	3,1
B	ADX(20) > 25	30 barras	2,4
C	ATR subiendo	6 barras	0,7
D	CCI(50) < −100	44 barras	−0,2

La generación siguiente se hace así:

- **Selección.** A y B, las mejores, tienen más papeletas para ser padres. D casi ninguna.
- **Cruce** (probabilidad 89 % en tu configuración). Se toma una parte de A y otra de B: por ejemplo «RSI(14) cruza 30 hacia arriba» con la salida de B, 30 barras.
- **Mutación** (20 %). A un hijo se le cambia algo al azar: el 14 del RSI pasa a 22, o el bloque RSI se cambia por otro de la lista.
- El hijo se prueba con un backtest. Si cumple los filtros de aceptación, entra en el databank.

Repetido 25 veces (25 generaciones), la población se va pareciendo cada vez más a sus mejores miembros. **Eso es lo que hay que retener: un genético no prueba 832.299 ideas, prueba muchas variaciones de las pocas que le fueron bien al principio.**

Islas. En vez de una población hay 20, que evolucionan por separado. Cada 10 generaciones un 3 % de cada isla emigra a otra. Sirve para que no todas acaben en la misma idea. SQX pone la isla en el nombre: `Strategy 4.13.338` es la isla 4, generación 13, número 338 (`knowhow/research/genetic-build-islands-are-the-sampling-unit.md`).

Sangre nueva y reinicios. Cada 10 generaciones se sustituye el 5 % peor de cada isla por estrategias recién sorteadas. Al terminar las 25 generaciones, o si la métrica se estanca, la evolución **vuelve a empezar de cero** y sigue así hasta que se agota el tiempo.

2.3 Aceptadas, guardadas y el databank lleno

Tres números distintos que conviene no mezclar:

- **Generadas:** todas las que SQX inventó y probó.
- **Aceptadas:** las que cumplieron los filtros de aceptación en algún momento.
- **Guardadas:** las que hay en el databank al acabar. El databank tiene un tope (`MaxStrategies` , 10.000). Cuando está lleno, una aceptada nueva sólo entra **echando a la peor** según la métrica.

Ejemplo ilustrativo con un databank de 3: están guardadas tres estrategias con Ret/DD 5, 4 y 3. Llega una aceptada con Ret/DD 3,5: entra y sale la de 3. Llega otra con 2: cumple los filtros, pero no entra. Al final el databank no contiene «las que cumplen los filtros», contiene **«las mejores por la métrica»**, y el listón de entrada ha subido solo.

2.4 Qué es un bloque y qué es una paleta

Un **bloque** es una pieza de regla: «RSI cruza un nivel hacia arriba», «ATR subiendo», «el cierre está por encima de la banda superior de Keltner». SQX tiene tres clases en la tarea Build (`sqx/projects/buildcensus.py:18`):

clase en SQX	qué es	cuántos hay en la tarea
`signals`	una **condición** (verdadero o falso)	575, más 23 de reloj
`indicators`	un **valor** que se compara con otro	161
`stopLimitBlocks`	el **precio** de una orden stop o limit	85

En total, 844 entradas en la tarea; la taxonomía (`sqx/blocks/taxonomy.yaml`) las reduce a 767 bloques porque un mismo bloque puede jugar dos papeles.

Una **paleta** es la lista de bloques encendidos para un build. Todo bloque fuera de la lista se apaga y SQX no puede sortearlo. La doctrina del dueño (regla 15): nunca toda la lista, una paleta por familia de idea (tendencia, ruptura, reversión…).

Por qué importa el tamaño de la paleta. Ejemplo ilustrativo: si una estrategia lleva una condición de entrada elegida entre 100 bloques, hay 100 formas de regla. Si lleva una o dos elegidas entre 100, hay 100 + 4.950 = 5.050 (4.950 son las parejas sin repetir: 100 × 99 / 2). Con 400 bloques son 400 + 79.800 = 80.200. Multiplicar la lista por 4 multiplica las formas por 16. Y cada forma admite miles de combinaciones de números (periodos, niveles, barras de salida). El apartado 3.5 hace la cuenta con la corrida real.

2.5 La consecuencia estadística: el número de ensayos

Esta es la única idea estadística del paso, y todo lo de después se lee contra ella.

El juguete. Tira una moneda 100 veces. Sacar 60 caras o más es raro: pasa el 2,8 % de las veces. Si alguien te enseña una moneda con 60 caras, sospecharías que está trucada.

Ahora tira **1.000 monedas**, 100 veces cada una, y quédate con la mejor. La mejor saca, de media, **66 caras** (entre 64 y 69 en nueve de cada diez repeticiones; simulado por mí, 2.000 repeticiones, `trials.py`). Ninguna moneda está trucada. Lo raro dejó de ser raro porque buscaste entre mil.

Lo mismo con estrategias. El resultado de una estrategia sin ninguna ventaja, medido con una t (beneficio medio por operación dividido por su error), cae alrededor de cero: por encima de 2 sólo el 2,3 % de las veces. Pero si pruebas N estrategias sin ventaja y miras la mejor:

N ensayos independientes sin ventaja	t esperada de la mejor	cuántas superan t = 2 por suerte	cuántas superan t = 3
1	0	0	0
100	2,53	2	0
10.000 (las guardadas)	3,86	228	13
47.976 (las aceptadas)	4,23	1.091	65
832.299 (las generadas)	**4,83**	**18.935**	**1.124**
4.880.561 (todo lo generado en esta celda)	5,17	111.033	6.588

Calculado por mí (`trials.py`): fórmula del máximo esperado de N normales de Bailey y López de Prado, comprobada por simulación hasta N = 10.000 (3,85 simulado frente a 3,86). Supuesto fuerte: ensayos independientes. Las dos columnas de la derecha valen sólo para ensayos independientes; con parientes no se pueden leer.

Segundo supuesto: t normal. Con las operaciones reales de la corrida (cola gruesa, curtosis en exceso mediana 3,5, pero asimetría positiva) la cola alta de la t sale algo más fina que la normal: P(t > 3) = 0,00090 frente a 0,00135 (calculado por el revisor remuestreando con media cero las operaciones de 150 estrategias, 600.000 t simuladas; el remuestreo no conserva la dependencia en el tiempo). Por este lado la tabla peca de prudente.

Qué quiere decir para ti. Que una estrategia del databank tenga t = 4 en el `build` no mide cuánta ventaja tiene: entre 832.299 intentos sin ninguna ventaja, la mejor tendría 4,83. En esta corrida la mediana de t pasa de 2,61 en el `build` a 0,74 en `oos1`, y de 8.514 con t > 2 en el `build` quedan 722 en `oos1` (calculado por el revisor, t = media / desviación × √n sobre las operaciones de cada tramo). El orden sí conserva algo: el quinto de mejor Ret/DD del `build` da 21 de las 72 vivas y el quinto peor 8 (Spearman +0,39 entre Ret/DD del `build` y t en `oos1`; calculado por el revisor; es un lote, USDJPY subió en `oos1` y los parientes hacen que 10.000 puntos no sean 10.000 observaciones). Sirve para ordenar candidatas, no para creer en una. Por eso el `build` no sirve para creer en una estrategia, sólo para proponerla, y por eso el paso 8 la juzga en `oos1`, que el generador no vio.

El matiz que lo complica. Las 832.299 **no son independientes**: son parientes (apartado 2.2). El número de ensayos «de verdad distintos» es menor, y nadie lo ha medido para el build. La ficha de las islas mide algo relacionado al otro extremo del embudo: las 217 supervivientes al paso 8 de cinco lotes equivalen a unas 5,5 apuestas independientes. Si los ensayos fueran un sorteo, 832.299 sería el techo. No lo son: el genético elige cada estrategia mirando las anteriores, y una búsqueda dirigida puede ajustarse al `build` más que un sorteo del mismo tamaño. No hay prueba de que 832.299 sea techo. En esta corrida la t más alta del databank en el `build` es 4,65, por debajo de 4,83; y hay 2.581 guardadas con t > 3 y 64 con t > 4, frente a 1.124 y 26 que darían 832.299 ensayos independientes, porque las buenas vienen repetidas (todo calculado por el revisor, t = media / desviación × √n sobre las operaciones del `build`). Un orden de magnitud de error en N mueve la t esperada unas 0,4 (3,26 con mil ensayos, 3,86 con diez mil, 4,83 con 832.299, 5,30 con diez millones; calculado por el revisor), pero los recuentos son proporcionales a N. Para las guardadas sí hay una medida (calculado por el revisor sobre el P/L diario del `build`, 2.632 días): las 10.000 de B5 equivalen a entre 10 (razón de participación de los autovalores) y unos 180 (factores para el 90 % de la varianza) resultados independientes; la correlación media entre dos es +0,22. Para las 832.299 generadas no la hay.

---

3. Lo que hace, pieza a pieza

Para cada pieza: la regla, dónde está y lo que quedó escrito en la tarea Build de la corrida (`Build-Task3.xml` del `project.cfx` archivado, leído con `sqx.projects.buildcensus.census`).

3.1 La forma de la estrategia (`assets/_build.yaml`)

ajuste	valor cargado	qué efecto tiene	código	en la corrida
`complexity.max_entry_conditions` / `max_exit_conditions`	2 / 2	tope de condiciones por lado (regla 14)	`buildrules.what_to_build` (`:116`)	entrada 0..1, salida 0..2
`complexity.simple`	entrada 1..2, salida 1..2	lo que se escribe sin plantilla	`buildrules.py:134`	no aplica (hay plantilla)
`complexity.lookback_bars`	1	mínimo y máximo a la vez: sólo la última barra cerrada	`buildrules.py:185`	`minShift="1"` `maxShift="1"`
`order_types`	`[EnterAtMarket]`	apaga todas las demás órdenes de entrada	`buildrules.set_order_types` (`:47`)	sólo `EnterAtMarket`
`exits.bars`	de 2 a 24 **horas**	salida por barras obligatoria (probabilidad 100); se pasa a barras según el timeframe	`buildrules.set_exit_types` (`:81`), `bar_range` (`:66`)	de 4 a 48 barras
`exits.by_condition` / `condition_required`	sí / no	salida por regla disponible al 50 %	`buildrules.py:101-105`	`_ExitRule_` al 50 %
`exits.stop_loss` , `profit_target` , `trailing_stop` , `move_sl_to_be`	no	sin stop, objetivo, trailing ni break-even	`set_exit_types` , `set_slpt` (`:198`)	los once interruptores de SL/PT en `false`
`money_management`	riesgo fijo de 1.000 USD por operación, tamaño por ATR(20) × 4, capital 100.000, drawdown máximo 30 %	idéntico en todas las tareas	`tasksettings.set_money_management` (`:106`)	`ATRRiskBasedSizingFixedRisk` , 100.000, 30
`trading_options`	cierre el viernes a las 21:00	idéntico en todas las tareas	`tasksettings.set_params` (`:60`)	`ExitOnFriday true` , `75600`
`engine`	MetaTrader5 (hedged)	motor del backtest	`tasksettings.set_engine` (`:30`)	igual
`precision.build`	1 (timeframe elegido)	fidelidad del backtest dentro de cada barra	`tasksettings.set_precision` (`:43`)	`testPrecision="1"`
`crosschecks.build`	`[MonteCarloManipulation]`	qué comprobación extra corre por estrategia	`tasksettings.set_crosschecks` (`:133`)	encendido, 30 simulaciones, sus dos condiciones apagadas
`databank.stop_condition`	`time-limit`	el build para por tiempo	`tasksettings.set_databank_caps` (`:186`)	`time-limit` , 1 h 30 min
`databank.caps`	`Trade_` y `Research_` : 10.000 y 180 min; `Test_` : 300 y 10 min	topes por prefijo si no se dan por argumento	`doctrine.caps` (`:155`)	10.000 y 90 min (por argumento)
`databank.max_strategies`	10000	**ninguno: ningún módulo lo lee**	—	—

La conversión de horas a barras, hecha a mano. En M30 hay 2 barras por hora: 2 horas × 2 = 4 barras, 24 horas × 2 = 48. En H1 serían 2 y 24; en H4, 1 y 6 (nunca menos de una barra, `buildrules.py:78`).

El recuento de condiciones con plantilla (`buildrules.py:137-140`). SQX aplica el rango «What to build» a **cada hueco aleatorio**, encima de lo que la plantilla fija. La plantilla `doubleROCLong` lleva en la entrada una condición fija y un hueco libre, y nada en la salida. La cuenta: tope 2 − 1 fija = 1, así que el hueco se escribe **0..1**. En la salida no hay nada fijo: se escribe 0..2.

Dos precisiones sobre ese ejemplo:

- La «condición fija» de `doubleROCLong` son en realidad **dos** bloques (ROC > nivel en dos periodos). Por decisión tuya del 2026-10-05, la misma condición en dos periodos cuenta como una. Así que la estrategia más larga de esta corrida tiene tres bloques en la entrada: ROC, ROC y el del hueco.
- La salida 0..2 **no produce nada** aquí: la plantilla no tiene hueco en la salida, y el rango se aplica a huecos (apartado 5.4).

3.2 La búsqueda genética (`sqx/projects/buildmode_model.xml`)

El bloque `<BuildMode>` se copia literal del fichero modelo. No se parametriza desde `_build.yaml`: es tu configuración del 2026-10-02 («pon 250 por isla, default ya para todo»). El significado de cada ajuste es el de la interfaz de SQX; no lo he decompilado.

ajuste	valor	qué hace
`generationType`	`genetic-evolution`	genética. `random-generation` sólo con `builder --random`
`PopulationSize`	250	estrategias por isla
`Islands`	20	poblaciones que evolucionan por separado
`MaxGenerations`	25	generaciones por evolución
`CrossoverProbability`	89	% de hijos que nacen de cruzar dos padres
`MutationProbability`	20	% de hijos a los que se cambia algo al azar
`MigrationModulo` / `MigrationRate`	10 / 3	cada 10 generaciones emigra el 3 % de cada isla
`FreshBloodWeakestPct` / `…Generations`	5 / 10	cada 10 generaciones se cambia el 5 % peor por sorteadas nuevas
`FreshBloodReplaceSimilar`	sí	también se sustituyen las demasiado parecidas
`DecimationCoef`	1	la población inicial no se «diezma» (no se sortea de más para quedarse con las mejores)
`<Conditions />`	vacío	ninguna condición para entrar en la población inicial
`EvoRestartOnFinish`	sí	al acabar las 25 generaciones, vuelve a empezar
`EvoRestartOnStagnation`	sí, 5 generaciones, `fitnessType="11"`	reinicia si la métrica se estanca durante 5 generaciones, medida sobre «In sample (Training)». `fitnessType` es un tipo de muestra: 10 = «In sample (whole)», 11 = «In sample (Training)», 40 = «In sample (Validation)»; de fábrica es 10 y 30 generaciones (`internal/plugins/TaskBuild/task_forex.xml:90` ; `SettingsGeneticOptionsService.js:198-207` ; `knowhow/conditions/build-rankings-acceptance.md`). Tú lo cambiaste a 11 y a 5
`EvoInSamplePeriod ratio`	50	reparto Training/Validation del tramo IS; 50 es el valor de fábrica (`task_forex.xml:91`)
`EvoStagnationRestartGenerations` 3	—	**no decodificado** (no sale en la plantilla de fábrica ni en el JS del servicio). Queda sin decodificar también qué mitad es Training y si el fitness del reemplazo usa Training o el IS entero

La cuenta de una evolución, ilustrativa. 20 islas × 250 = 5.000 estrategias por generación; × 25 generaciones = 125.000 por evolución completa. En 90 minutos SQX generó 832.299, unas 6,7 veces esa cifra. Eso daría unas 3 a 7 evoluciones completas, según se cuente el número real por generación (494 en los nombres frente a 250 de población: con 494, una evolución son al menos 20 × 494 × 25 ≈ 247.000 candidatos nombrados, o sea no más de 3,4 evoluciones; la lectura de 7 no cuenta los candidatos sin nombre; cuenta del revisor). Los reinicios no están observados en ningún log ni dato. No es una cuenta exacta y no sé por qué el número de orden llega a 494.

Genética frente a aleatoria. El comando único construye siempre en genética: `bin/workflow.sh` no tiene `--random` . La gemela aleatoria existe (`builder --random` , `patch.set_generation`) y `set_genetic` respeta ese atributo al copiar el modelo (`tasksettings.py:181-182`). **No hay ninguna comparación vigente entre las dos:** la que había (dossier del 2026-10-02, «la genética da de 3 a 18 veces más buenas por minuto») salió de la calibración borrada.

3.3 La métrica que optimiza

La tarea lleva `<Ranking type="ReturnDDRatio" />` : beneficio neto dividido por el drawdown máximo, **del tramo** `build` . Es a la vez lo que guía al genético y el criterio de reemplazo del databank lleno.

Fórmula, comprobada en el export: Ret/DD = beneficio neto / drawdown máximo en dinero. En las 10.000 de la corrida, `Net profit (IS)` / `Drawdown (IS)` coincide con `Ret/DD Ratio (IS)` con una diferencia máxima de 0,005 (el redondeo a dos decimales; calculado por mí, `databank.py`). Ejemplo, una fila real (`Strategy 10.22.339`): 45.550,26 USD de neto con 4.128,05 USD de drawdown da 11,03.

Ningún módulo la escribe (no hay ningún `Ranking type` en `sqx/` , `core/` ni `pipeline/` ; `buildcensus.py:101` sólo la lee). Es la que traía el donante.

Qué dice la ficha `knowhow/research/ga-target-choice.md` **:**

- La elección de métrica **está sin resolver**. Cinco corridas de 10.000 en XAUUSD (2026-09-05): repetir la **misma** métrica dos veces cambió el resultado en 16 puntos; cambiar de métrica, entre −1,7 y +6,5 puntos. El ruido tapa el efecto.
- Dentro de la muestra las métricas se ordenan casi igual: Ret/DD frente a Sharpe, ρ = +0,98; profit factor, SQN y R Expectancy son la misma (ρ = +1,00). En la corrida B5 la relación es más floja: Ret/DD frente a Sharpe ρ = 0,87, frente a profit factor 0,72 (Spearman, calculado por mí sobre las 10.000 guardadas, que ya están cortadas por Ret/DD).
- Recomendaba seleccionar **después** del build, no dentro.

Aviso sobre esa ficha: su evidencia es de septiembre, anterior al cambio de feeds y de costes del 2026-10-01. Vale como «no está resuelto», no como cifra.

3.4 Los filtros de aceptación (`assets/_study.yaml`) — PROVISIONALES

`core/buildfilters.py` resuelve los umbrales (por defecto ← clase del activo ← activo) y `rankings.set_rankings` (`sqx/projects/rankings.py:19`) **sustituye enteras** las condiciones del donante (más de 500 operaciones, Sortino > 0,6, Stagnation < 540).

Las cuatro condiciones, todas medidas sobre el `build` (`sampleType="10"` ; `sample: IS` es el único valor admitido, `buildfilters.py:14` y `:61`):

condición	fórmula	umbral cargado	en la corrida (USDJPY M30, 10 años)
suelo de operaciones	`NumberOfTrades ≥ techo_hacia_arriba(mín/año × años)`	10 al año	≥ 100
techo de operaciones	`NumberOfTrades ≤ suelo_hacia_abajo(máx/año × años)`	75 al año (H1 y M30), 100 (H4)	≤ 750
profit factor	`ProfitFactor ≥ valor`	1,0	≥ 1,0
beneficio neto	`NetProfit > valor`	0	> 0

Las cuentas a mano (`buildfilters.py:63-78`). Años del `build` = días entre el principio y el final / 365,25, a dos decimales. USDJPY: 10,0 años → suelo 10 × 10 = 100, techo 75 × 10 = 750. USA500 tiene un `build` de 7,95 años y un suelo propio de 13 al año: 13 × 7,95 = 103,35 → se redondea hacia arriba, 104; techo 75 × 7,95 = 596,25 → hacia abajo, 596. Coincide con lo que el registro de proyectos guardó para `Test_USA500_H1_indicesIBS` (`NumberOfTrades >= 104 & <= 596`).

Profit factor, desde cero: suma de las ganancias dividida por la suma de las pérdidas. Si las operaciones ganadoras suman 300 y las perdedoras 200, el profit factor es 1,5 y el neto 100. **Un neto mayor que cero y un profit factor mayor que 1 son prácticamente la misma condición**: sólo difieren en un profit factor de 1,0 exacto (neto 0), que la de neto > 0 ya excluye; la tercera y la cuarta fila del cuadro se repiten.

Excepciones cargadas: índices y crudo llevan un suelo por año más alto (13, 15 o 16) porque su `build` es más corto, para que el total no baje de unas 100 operaciones. Un timeframe que no esté en el fichero no se puede construir.

Modo calibración (`builder --acceptance calibration`): sólo «≥ 100 operaciones y neto > 0», y apaga el `MonteCarloManipulation` (`rankings.py:69-70`). Nunca para un `Trade_` , dice el fichero; el código no lo impide.

Cuánta población dejan pasar en la corrida real:

	USDJPY M30 `doubleROCLong` , lote B5
generadas	832.299
aceptadas	47.976 (5,76 %)
rechazadas	784.323
guardadas	10.000 (el 20,8 % de las aceptadas)

Y en los cinco lotes de la misma celda y la misma configuración (filas del ledger):

lote	generadas	aceptadas	%	vivas tras el paso 8
`…_10k`	832.137	49.189	5,91	56
`…_B2`	749.129	49.200	6,57	48
`…_B3`	777.781	50.155	6,45	10
`…_B4`	784.406	48.530	6,19	31
`…_B5`	832.299	47.976	5,76	72
total	**3.975.752**	**245.050**	6,16	**217**

La aceptación es estable (entre 5,8 y 6,6 %). Lo que no puedo decir es **cuánto quita cada condición por separado**: SQX sólo escribe el total de rechazadas, y las demasiado parecidas a otra cuentan como aceptadas y se retiran después, antes de entrar en el databank (`knowhow/pipeline/blind-steps-write-no-ledger-rows.md:26-32` : 4.600 aceptadas, 4.238 en el databank). Qué más entra en «rechazadas» fuera de los filtros escritos no está documentado (`DismissTooSimilarStrategies` y `AutomaticDismissal` se heredan del donante). Por eso «guardadas = 20,8 % de las aceptadas» mezcla dos filtros: la similitud y el reemplazo por Ret/DD.

Lo que hay dentro del databank (10.000 guardadas, métricas del `build` ; calculado por mí, `databank.py`):

	mínimo	25 %	mediana	75 %	máximo
operaciones	100	163	269	586	750
profit factor	1,14	1,34	1,53	1,86	3,57
Ret/DD	**2,90**	4,27	5,75	7,77	22,96
beneficio neto (USD)	12.702	36.351	45.553	64.602	115.001

- Las 10.000 cumplen los cuatro filtros: SQX los respeta.
- El suelo y el techo de operaciones **sí muerden**: hay estrategias con 100 y con 750 exactas.
- El filtro de profit factor y de neto **no muerde**: la guardada de peor Ret/DD (2,90, `Strategy 10.6.400(1)`) tiene profit factor 1,19 y neto 50.889,94 USD con 663 operaciones; el profit factor mínimo del databank es 1,14 (`Strategy 18.12.265` , Ret/DD 3,48) y el neto mínimo 12.702, otra estrategia más (los mínimos de columna no son una fila; calculado por el revisor). Lo que dejó dentro a la de peor Ret/DD fue ese 2,90; todo lo aceptado por debajo de eso se quedó fuera por el reemplazo.
- **6.088 de las 10.000 operan menos de 35 veces al año** (menos de 350 operaciones en 10 años). El paso 8 no las tira todas: con menos de 35 deja en Limbo y sólo tira por debajo de 20 al año (`criteria.yaml:79-80`), unas 3.900 de las 10.000 (calculado por el revisor).
- La generación tardía pesa más: 13 estrategias de la generación 1 y más de 620 de cada una de las cinco últimas. La mediana de Ret/DD sube de 4,19 (generaciones 1-5) a 6,67 (21-25). Es el genético haciendo su trabajo sobre el `build` ; la ficha de las islas mide que la supervivencia al paso 8 va al revés (0,71 % por debajo de la generación 10, 0,26 % desde la 20).

3.5 La paleta y cuánto estrecha la búsqueda

Cómo se resuelve una paleta (`sqx/blocks/palette.py:141` , función `resolve`). Para cada uno de los 767 bloques de la taxonomía:

1. Si la paleta lo nombra en `overrides` , vale ese peso (0 lo apaga).
2. Si la paleta es una lista corta (`unlabelled: off` y tiene `overrides`), todo lo demás se apaga.
3. Si no, vale el peso de la taxonomía para la familia de la paleta (0 a 3).

Un bloque queda encendido si su peso es mayor que cero. La taxonomía tiene hoy los 767 bloques etiquetados (588 con papel de condición, 171 de valor, 85 de nivel).

Cómo se escribe (`sqx/projects/buildingblocks.py:37` , función `rewrite`). Recorre las 844 entradas de la tarea y pone `use` y `weight` . Dos cosas se apagan **diga lo que diga la paleta** (regla 15, `:62-64`):

- todo bloque de categoría `stopLimitBlocks` (el precio de una orden stop o limit);
- todo bloque de reloj o calendario, que reconoce por el nombre: `CurrentHour…` , `BarHour…` , `…DayOf…` , `…WeekOf…` , `…Month…` , `…Time…` , `IsMonth…` (`buildcensus.py:14`).

Un bloque que la taxonomía no conoce también se apaga. Varias paletas separadas por comas son su unión (`:19`).

La banda de tamaño. `palette.CONDITIONS = (90, 170)` (`palette.py:23`): menos de 90 condiciones y la búsqueda se ahoga, más de 170 y vuelve el sobreajuste. Es cifra tuya del 2026-09-24. El comando dice «dentro» o «FUERA», pero **no se niega** a aplicar una paleta fuera de la banda (`buildingblocks.py:109-110`).

La librería hoy (`python3 -m sqx.blocks.palette`): 20 paletas. Trece están dentro de la banda, entre ellas las seis `_v3` por familia (de 99 a 127 condiciones) y las cuatro de plantilla (`doubleROCLong_USDJPY_M30` 103, `squeezeBBLong_XAUUSD_H1` 112, `indicesIBS_USA500_H1` 136 y `timeRangeBreakout` 124). Siete están fuera, con 2 a 12 condiciones: las tres `_v2` y `momentum_base` , `patron_base` , `sesion_base` , `volatilidad_base` .

El agente `buildingBlocksExpert` (`.claude/agents/buildingBlocksExpert.md` , modelo Opus) elige la paleta de una plantilla: decide para qué sirve la condición libre dada la idea (un filtro de régimen para una ruptura, uno de agotamiento para una reversión…), deja fuera lo que duplica el bloque fijo, y escribe la paleta con una nota que lo razona. No toca SQX hasta aplicar la paleta con el custodio parado.

En la corrida real:

	encendidos	de
condiciones	**103**	575
valores / indicadores	0	161
niveles stop/limit	0	85
reloj y calendario	0	23

La paleta apaga el 82 % de las condiciones. Su nota explica el criterio: junto a un doble ROC positivo, el hueco se rellena con régimen de volatilidad (peso 3), disparo de reversión por el lado comprador y fuerza de tendencia o rango (peso 2); fuera toda la familia momentum, tendencia y ruptura (repiten la fija), patrones de velas, sesión y reloj.

El criterio de la paleta no es sólo doctrina: su nota cita medidas hechas sobre USDJPY M30 (`assets/FAMILIAS.md` : momentum en USDJPY M30 largo, «medido nota C, 3,3× el coste, no significativa tras corregir», y «la medida líder de la celda es una vela > 2 ATR»). La paleta se escribió el 2026-10-05 a las 21:25, dos horas antes del primer build de la celda en el ledger (23:40), así que no se eligió mirando builds anteriores; pero las medidas que la guían son miradas a los datos que el recuento de ensayos no suma (`ledger/report.py:99-100` las imprime aparte).

Cuánto estrecha, en formas de regla (calculado por mí; supuestos: sin contar parámetros, parejas sin repetir y sin orden, entrada y salida sortean de la misma lista):

caso	lista	formas de regla posibles
esta corrida: plantilla con un hueco 0..1 y sin salida por regla	103	1 + 103 = **104**
la misma plantilla sin paleta (lista del donante)	400	401
la misma plantilla con todo encendido	575	576
Simple Strategy 1..2 entrada y 1..2 salida, paleta de 103	103	5.356 × 5.356 = **28,7 millones**
Simple Strategy, lista del donante	400	80.200 × 80.200 = 6.432 millones

Dos lecturas:

- **Con una plantilla de un hueco, la paleta estrecha por 4** (104 frente a 401). Con una Simple Strategy, por 224. La paleta importa mucho más cuando no hay plantilla.
- **La paleta no cambia cuántas estrategias se prueban:** eso lo fija el tope de minutos (`stop_condition: time-limit` ; los cinco lotes generaron entre 749.129 y 832.299 con la misma configuración). Con 104 formas y 832.299 ensayos, cada forma se prueba con unos 8.000 juegos de números distintos (832.299 / 104; calculado por el revisor). Una paleta más corta concentra más intentos en cada forma; ninguna medida en disco dice si eso ajusta más o menos al `build` .
- **Encima de cada forma van los números.** Cada periodo se sortea entre 2 y 200 y la salida entre 4 y 48 barras. Las guardadas llevan entre 4 y 10 parámetros (8 la mayoría: 5.746 de 10.000).

Lo que de verdad se usó. El estudio guarda de cada estrategia una huella de su lógica sin los números (`core/sqxfile.py:110`). En las 10.000 hay **259 huellas distintas**; la más frecuente reúne 2.121 estrategias, la segunda 1.613, **5 huellas reúnen la mitad** y 49 el 90 % (calculado por mí, `structure.py`). Que salgan 259 y no 104 como máximo se debe a que la huella cuenta también bloques que el fichero arrastra sin usar: en tres estrategias de esta misma celda abiertas a mano, dos con la misma regla (ROC, ROC y `CBlock_WassersteinShiftHigh`) tenían huella distinta. Es decir, 259 **exagera** la variedad.

3.6 Lo que el build deja escrito

salida	dónde	contenido
databank «Results»	en SQX; exportado a `AlgoData/metrics/<P>/Results/metrics.csv`	las guardadas, con 38 columnas de métricas
operaciones	`AlgoData/raw/<P>/Results/<día>/trades.parquet`	3.496.906 operaciones del `build` en la corrida
fila del ledger	`AlgoData/ledger/<estudio>.jsonl`, `step: 6`	`n_in` = generadas, `n_out` = aceptadas, `n_scored` = generadas con unidad `generated`
registro de proyectos	`AlgoData/projects/registry.csv`	plantilla, activo, timeframe y la aceptación escrita en texto
resumen del piloto	`AlgoData/autopilot/<P>/<corrida>/resumen.md`	«CONSTRUCCION: 0 → 10000 · OOS: 0 → 10000»

La fila del ledger se lee de la línea que SQX escribe al acabar la tarea, de esta forma (ejemplo de otro build de la misma celda, cuyo log sigue en disco): `Strategies generated: 148217, Time per strategy: 6 ms., …, Accepted: 24005, Rejected: 124212, Strategies per hour: 592855`.

Tiempos de la corrida: build 90,1 minutos (el tope exacto), retest OOS de las 10.000 en 67 segundos, exportación 7 minutos. Ritmo: 9.240 generadas y 533 aceptadas por minuto. A ritmo constante el databank se llenaría hacia el minuto 19-20 (10.000 / 533 = 18,8 minutos suponiendo que toda aceptada entra; el ritmo del arranque es menor y entre aceptadas y guardadas hay un 8 % de parecidas); los unos 70 minutos restantes son reemplazo por Ret/DD.

---

4. El criterio cargado y por qué

4.1 No hay reglas de paso; hay negativas

`pipeline/autopilot/criteria.yaml` no tiene bloque para el paso 6, `pipeline/autopilot/facts.py` no extrae hechos de él y `ledger/thresholds.yaml` no tiene umbrales suyos. Lo que hay son negativas que paran antes de construir:

qué comprueba	regla	dónde
el nombre del proyecto	`Test_`, `Trade_` o `Research_`	`builder.py:207`
una sola dirección	la plantilla abre sólo largos o sólo cortos (regla 13)	`builder.directions` (`:34`), `:215`
el tope de condiciones	no más de un hueco libre por lado; fijas + hueco ≤ 2 (regla 14)	`buildrules.over_the_cap` (`:144`)
costes del activo decididos	nada pendiente en la ficha (regla 5)	`builder.py:222`
sesión y timeframe	la ficha tiene sesión; el timeframe se conoce	`doctrine.blockers` (`:63`)
el worker parado	no se escribe un `project.cfx` que una instancia tiene abierto (regla 4)	`builder.py:226`, `buildingblocks.py:85`
hay filtro para el activo y el timeframe	si falta un valor en `_study.yaml`, no se construye	`buildfilters._need` (`:18`)
el filtro lee sólo el `build`	`sample` distinto de `IS` se rechaza	`buildfilters.py:61`
una Simple Strategy lleva paleta	sin paleta no se lanza	`bin/workflow.sh:51`
la paleta no enciende reloj ni niveles	se apagan y se dice cuáles	`buildingblocks.py:62`

4.2 Por qué los filtros son tan bajos

El argumento escrito en `_study.yaml` es: un filtro del `build` dentro del build no da más estrategias robustas por minuto que no filtrar, y cortar después da lo mismo; así que dentro va lo mínimo y el corte se hace en Python, donde se puede ver qué se quitó. La ficha `knowhow/research/is-filters-inside-the-build.md` lo resume como «entre un 70 y un 98 % menos de robustas por minuto con cualquier filtro dentro».

Estado de esa evidencia: sin medida vigente. Sale de la calibración `Test_Calib_*` y de su segunda ronda, borradas el 2026-10-04 por el spread. Las carpetas que el fichero cita (`AlgoData/scratch/calib2/…`) ya no existen. Y la propia ficha avisa de que no se probó en builds largos con el databank lleno, que es justo el caso de trabajo.

4.3 Qué queda del dossier del 2026-10-02

`docs/AgentPDFs/criterios-pasos-6-y-8-2026-10-02.md`, sólo su parte del paso 6 (§4):

lo que decía	estado hoy
quitar los filtros del donante (más de 500 operaciones, Sortino, Stagnation)	**vigente como mecánica**: `rankings.py` los sustituye
suelo de 10 operaciones al año, PF ≥ 1,0, neto > 0	**cargado**, sin medida vigente
techo de 150 al año en H1, 200 en M30, 100 en H4	**ya no**: hoy 75, 75 y 100 (propuesta del 2026-10-04, también de la calibración borrada)
la tabla por activo (100-2.000 en M30, etc.)	**no vale**
la prueba A/B de filtros dentro del build, las «buenas por minuto»	**no vale** (calibración borrada)
la tabla del suelo de operaciones y la potencia	**no vale**
«genética: de 3 a 18 veces más buenas por minuto»	**no vale**
«el objetivo del genético se queda en Ret/DD»	sigue siendo lo cargado; como conclusión medida, **no vale**
«máximo de 3 condiciones no aporta»	**superado** por tu regla 14 (máximo 2)
aviso: un build largo llena el databank y luego sustituye por Ret/DD, sin medir	**vigente y sin resolver** (apartado 5.2)
`oos1` de USDJPY y XAUUSD gastado al calibrar	el hecho ocurrió aunque los datos ya no estén

4.4 Si mueves un ajuste

Sin medida vigente no puedo dar el coste en estrategias de cada cambio; sí lo que se puede leer del databank de la corrida, que es sólo aritmética sobre lo guardado:

- **Subir el suelo a 35 operaciones al año dentro del build:** de las 10.000 guardadas hoy, 6.088 no entrarían. No quiere decir que el databank tendría 3.912: seguiría llenándose hasta 10.000 con otras, que nadie ha visto.
- **Subir el profit factor mínimo a 1,14 o menos, o exigir Ret/DD ≥ 2,9 o menos:** no excluiría ninguna de las 10.000 actuales; el efecto sobre la búsqueda no está medido (el filtro también decide qué se acepta y qué cría el genético por el camino: 47.976 aceptadas, no sólo las 10.000).
- **Bajar el tope de minutos:** con unos 19-20 minutos el databank ya estaría lleno, a ritmo constante. Lo que compran los minutos de más es reemplazo por Ret/DD del `build`, y eso no está medido (apartado 5.2).

---

5. Lo que no cuadra o no está medido

5.1 El número de ensayos existe y casi nadie lo ve

SQX generó 832.299 y el ledger lo guarda (`ledger/steprows.py:40-58`). Pero:

- el resumen del piloto escribe «CONSTRUCCION: 0 → 10000»;
- el informe de lotes dice «Generadas acumuladas: 50,000» y titula la columna «generadas» con las 10.000 guardadas de cada lote (`studies/screening/batches/report.py:43`, `:53`); las generadas de verdad en esos cinco lotes son 3.975.752;
- la ficha `knowhow/sqx-drive/unselected-build-population.md` dice, en inglés, que el build de trabajo es el top 10.000 por fitness del IS de un número mayor que no se apunta; desde el 2026-10-04 (commit `d8e1ffe`, `log_build`) sí se apunta (la primera fila `step 6` del ledger de la celda es del 2026-10-05);
- la cifra del informe de lotes es 83 veces menor que la del ledger (832.299 / 10.000; calculado por el revisor). Hay un tercer sitio: el almacén de supervivientes guarda en `runs.json` un campo `generated` que es el número de estrategias que juzgó el gate (10.000) y lo llama denominador (`core/store/save.py:82`, `core/store/write.py:196`); y las filas de calibración del ledger apuntan como `generated` las estrategias del databank (`ledger/steprows.py:89`). Hoy ninguna corrección lee ninguna de esas cifras; el día que una lo haga, leería 83 veces menos búsqueda de la que hubo;
- **ninguna regla de los pasos 8 a 25 consume ninguna N.** Las reglas del paso 8 comparan cada estrategia con umbrales fijos (t ≥ 1,65), sin corrección por cuántas se juzgan; la `k` de `t_exceso_k` es el factor de coste 1,25 (`studies/screening/gate/config.yaml:78`), no un número de ensayos. `trials.accumulated` (`ledger/trials.py:13`) suma esa N y la llaman `ledger/report.py:92`, la ventana (`ui/daemon/ops/ledgerview.py:55`), el archivo (`core/archive/manifest.py:76`) y `portfolio/common/construct/oos2.py:63`; `trials.overall` (`ledger/trials.py:74`) la llaman `ledger/report.py:103` y `oos2.py:27`, y `oos2.py:69` pasa esa N a `deflate.read`, con lo que el veredicto oos2 de la cartera desinfla el Sharpe con una N total que ya incluye las filas «generated» del paso 6 (`ledger/trials.py:43`). Es decir, la N inflada del build ya entra en oos2. Las 832.299 y las 10.000 responden a preguntas distintas: cuánto está inflado lo que enseña el `build` y cuántas pruebas se hacen en `oos1`.

5.2 El filtro de verdad es el databank lleno, y no está medido

Desarrollado en 2.3 y 3.4: el corte efectivo de la corrida es Ret/DD ≥ 2,90 sobre el `build`, que nadie escribió. Es una selección por el tramo que el generador ya exprimió, aplicada durante unos 70 de los 90 minutos. Tú ya resolviste el 2026-10-07 que los 90 minutos se quedan (`knowhow/sqx-drive/unselected-build-population.md:27`); los 180 de `Trade_` y `Research_` (`_build.yaml:82-83`) son otro caso. Está apuntado como «sin probar» en `OPEN.md` #93 desde el 2026-10-02 y sigue igual. La única señal en disco apunta en contra de alargar: la supervivencia al paso 8 cae con la generación (ficha de las islas), pero es una observación sobre cinco lotes de una celda, no una prueba del tope de minutos.

5.3 La métrica la puso el donante

`ReturnDDRatio` no sale de `_build.yaml` ni de ningún módulo. Si un día cambiara el donante, cambiaría la métrica sin que nada avisara. Y la ficha que debería respaldarla dice «sin resolver».

5.4 La salida por condición es «optativa» en el papel e imposible con estas plantillas

`_build.yaml:44-46` dice que el generador «puede ponerla o no». `buildrules.py:137-140` escribe 0..2 en la salida cuando la plantilla no fija nada ahí. Pero el rango se aplica a huecos, y las plantillas de la librería no tienen hueco en la salida (`knowhow/authoring/condition-count-simple-vs-template.md`: «nothing on exit»). Resultado medido en la corrida: 3.043.763 cierres por barras, 453.143 por el viernes y **cero por regla**. Tres estrategias de la misma celda abiertas a mano sólo tienen las reglas «Trading signals» y «Long entry». La salida por condición sólo existe hoy en una Simple Strategy, donde es obligatoria (1..2).

5.5 El crosscheck del build y su comentario

`_build.yaml:102-106` explica el crosscheck de alta precisión («sólo en la construcción», «con el mismo spread»). La línea 108 enciende otro: `MonteCarloManipulation`. En la tarea de la corrida, `RetestWithHigherPrecision` está apagado y `MonteCarloManipulation` encendido con 30 simulaciones y sus dos condiciones de aceptación en `use="false"`. `_study.yaml:67` reconoce para el modo calibración que «no filtra nada y gasta CPU»; en el modo normal se deja encendido. No he medido cuánto tiempo cuesta. El comentario de `_build.yaml:106` promete que otro crosscheck «se enciende sólo si el dueño lo pide»; el commit que enciende `MonteCarloManipulation` en el build es `e53f703` (2026-09-28, de la ventana) y no consta que se pidiera.

Consecuencia: `_build.yaml:91-93` dice que el build se apoya en el crosscheck de alta precisión para descartar lo que sólo existe en las barras grandes. Ese crosscheck está apagado y el que está encendido no tiene condiciones. Hoy el build no hace ninguna comprobación de robustez: filtra por número de operaciones, por neto positivo y por Ret/DD del `build`, y nada más.

5.6 Comentarios de `_build.yaml` que contradicen sus valores

- **Precisión.** Las líneas 92-94 dicen «de ahí en adelante ya no hay atajo: todo a 1 minuto», y las 96-100 fijan `default: 1` (timeframe elegido). La línea 388 dice que el SPP «diverge de `precision.default: 2`», que ya no es 2. El docstring de `tasksettings.set_precision` (`:51-54`) repite lo del minuto.
- **`databank.max_strategies: 10000`** (línea 76) no lo lee nadie; los topes salen de `caps` o de los argumentos.
- **Precisión (otra línea).** `_build.yaml:249` dice `precision: 1 # un minuto — lento y fino`, pero 1 significa el timeframe elegido (la línea 242 lo dice bien).
- **Money management.** El comentario dice «debatible entre 3, 4 y 5» para el multiplicador del ATR. Cargado 4. Es decisión tuya abierta, no un error.

5.7 `_study.yaml` cita evidencia borrada y lleva un filtro que su dossier descartó

El encabezado remite a `criterios-pasos-6-a-12-2026-10-04.md` §4 y a `AlgoData/scratch/calib2/adaptive/RECIPE.md`. La carpeta no existe. Ese mismo dossier, en su §4.1, dice «profit factor: ninguno»; el fichero lleva `profit_factor_min: 1.0`. No hace daño (repite «neto > 0», apartado 3.4), pero no es lo que el dossier decía haber escrito.

5.8 La banda de la paleta no se hace cumplir, y dos textos la describen distinto

`palette.CONDITIONS` es 90-170; los nombres de las `_v3` dicen «90-150». El aplicador avisa y sigue. Siete paletas de la librería tienen entre 2 y 12 condiciones y se pueden aplicar.

Otros textos desfasados alrededor de la paleta:

- la cabecera de `taxonomy.yaml` dice «un bloque sin etiquetar no entra en ninguna paleta»; `palette.py:15-18` y `:168` lo dejan entrar a peso 1 con `unlabelled: neutral`. Hoy no hay ninguno sin etiquetar;
- el agente `buildingBlocksExpert` habla de «las tres paletas» (hay 20) y enumera seis familias diciendo siete (falta `session`);
- la nota de la paleta de la corrida dice que la estrategia «sale por SL/PT/barras de la plantilla»; la doctrina apaga SL y PT y en los datos sólo hay salidas por barras y por el viernes.

5.9 «Una paleta por familia» se cumple por costumbre en el camino con plantilla

Una Simple Strategy no se lanza sin paleta. Con plantilla, el argumento `-` deja la lista del donante (400 condiciones, 26 valores y 10 niveles encendidos). El comentario de `bin/workflow.sh:13-14` dice que `-` es para la plantilla sin hueco, pero **no he encontrado que nada compruebe que la plantilla no tiene hueco**. Los pesos de la paleta, además, SQX sólo los respeta entre bloques con nombre, no en los comparadores genéricos (`knowhow/authoring/builder-block-switches.md`); por eso las `_v3` no llevan valores.

5.10 Qué cuenta como «bloque de reloj»

La regla 15 se aplica por el nombre del bloque (23 en la tarea). `WORKFLOW.md` deja abierta la pregunta de los que leen la sesión o una franja sin llamarse así (`SessionHigh/Low`, `HighestInRange`, `CBlock_TimeBreakout*`). En la paleta de la corrida no he visto ninguno de esos entre las 103.

5.11 Las estrategias de un build no son sorteos independientes

Dicho en 2.5 y medido por la ficha de las islas: los recuentos por destino se dispersan entre islas 38 veces lo que daría un sorteo, y una sola isla dio 19 de 56 supervivientes en el primer lote (23 de 72 en B5). Para este paso significa dos cosas: que «10.000 estrategias» es mucho menos que 10.000 ideas (apartado 3.5), y que el número de ensayos efectivo de las generadas está sin medir. Para las guardadas sí hay una medida (calculado por el revisor sobre el P/L diario del `build` de las 10.000 de B5): equivalen a entre 10 y unos 180 resultados independientes (ver 2.5). Concuerda con las 5,5 apuestas de la ficha de las islas.

5.12 Lo que no está medido

- Si un build de 90 o 180 minutos con el databank lleno da más supervivientes que uno corto, o sólo estrategias más ajustadas al `build`.
- Genética frente a aleatoria con los costes de hoy.
- La métrica del genético, con réplicas.
- Cuánto rechaza cada filtro por separado y cuánto rechaza «demasiado parecidas».
- El coste en tiempo del `MonteCarloManipulation` del build.
- El número efectivo de ensayos de un build.
- El efecto de la paleta: no hay ningún par «misma celda, con y sin paleta» en disco.
- Ninguna corrida con Simple Strategy hecha con el código de hoy me ha servido de ejemplo; todo lo de 3.5 sobre ese caso es cuenta, no dato.

5.13 Dónde este documento corrige a los anteriores

- Dossier del 2026-10-02, §4: sus cifras no valen (apartado 4.3).
- Dossier del 2026-10-04, §4.1: dice que no queda filtro de profit factor; queda uno, redundante.

- Informe de lotes de la corrida: «generadas» son las guardadas.
- Una primera versión de este documento decía que el suelo de 35 operaciones al año se aplica en el paso 8: no es así, el paso 8 deja en Limbo entre 20 y 35 y tira por debajo de 20 (`criteria.yaml:79-80`).

5.14 Las «72 vivas» no llevan la vara del azar que hay en disco

El control de monos de esta celda (`falsePositives/funnel.csv` , fila `nula,8` , primer lote, 2026-10-07) pasó 5.000 estrategias de entrada aleatoria por el mismo `criteria.yaml` : 33 cruzan el paso 8 (2 Pasa y 31 Limbo), el 0,66 %. Sobre 10.000 serían unas 66, entre 45 y 93 (calculado por el revisor, Clopper-Pearson, sin corregir que los 5.000 monos salen de 100 donantes). Los cinco lotes reales dieron 56, 48, 10, 31 y 72 (media 43); la población real del primer lote da 58 de 10.000 con la cascada apagada (calculado por el revisor). Límites: los monos no pasaron por la selección del build, vienen de otro lote y `criteria.yaml` es provisional. El 0,0087 % que antes aparecía en la tabla de la sección 0 dividía por un número que no es el denominador de esa cifra.

5.15 Otros sitios que llaman «generadas» a las guardadas

Además del informe de lotes (apartado 5.1): `core/store/save.py:82` y `:151` definen `generated` como «cuántas estrategias juzgó el gate», `core/store/write.py:196-198` lo guarda en `runs.json` como «el denominador del que salieron las estrategias guardadas», y `ledger/steprows.py:89-102` (`calib_rows`) escribe las estrategias del databank con unidad `generated` . Es un dato guardado, no sólo un informe. Hoy ninguna corrección por pruebas múltiples bebe de ninguno (los `fdr` de `studies/screening/` y el paso 20 cuentan sus propias familias).

5.16 Qué otras miradas a los datos hay detrás de la paleta

Descrito en 3.5: la nota de la paleta cita medidas de `assets/FAMILIAS.md` sobre USDJPY M30, hechas antes del primer build de la celda y que el recuento de ensayos no suma. No es una elección ciega; tampoco es una elección sin miradas previas.

---

6. Cierre

6.1 Lo cargado, en una tabla

pieza	valor	estado
forma de la estrategia	una dirección, a mercado, salida por barras 2-24 h, sin SL/PT, ≤ 2 condiciones por lado, última barra	tus reglas duras 13, 14 y 15
búsqueda	genética, 20 × 250, 25 generaciones, reinicios	tu configuración del 2026-10-02
métrica	Ret/DD del `build`	heredada del donante; sin resolver
filtros	10-75 operaciones al año, PF ≥ 1,0, neto > 0	PROVISIONALES, sin medida vigente
topes	10.000 estrategias; 180 min (`Trade_` , `Research_`), 10 min (`Test_`)	tuyos, 2026-10-01
paleta	90-170 condiciones, sin reloj ni niveles	tuya; la banda no se hace cumplir

6.2 El resultado sobre la corrida real

832.299 generadas → 47.976 aceptadas → 10.000 guardadas (corte efectivo: Ret/DD ≥ 2,90 en el `build`) → 72 vivas tras el paso 8 (el 0,72 % de las 10.000 juzgadas; el azar sin ventaja deja unas 66, entre 45 y 93; calculado por el revisor). Las 10.000 son, como mucho, 104 formas de regla; cinco huellas reúnen la mitad; el 61 % opera menos de 35 veces al año (el paso 8 tira sólo las de menos de 20: unas 3.900).

6.3 Las decisiones que te quedan

#	decisión	lo que ganas	lo que cuesta
1	**Qué número de ensayos se enseña en el resumen y en el informe de lotes:** hoy dicen «10.000» (las guardadas), el ledger guarda 832.299 (las generadas).	Que el resumen y el informe de lotes hablen del mismo número. Hoy no cambia ningún veredicto del paso 8, porque ninguna regla lo lee; sí entra en el veredicto oos2 de la cartera (apartado 5.1). Las dos cifras miden cosas distintas: 832.299 es cuánto se buscó en el `build` (la mejor de 832.299 sin ventaja tendría t ≈ 4,8 en el `build`); 10.000 es cuántas se prueban en `oos1`.	No es elegir una de las dos: cada una responde a su pregunta. Las generadas son parientes y no son un techo demostrado; las guardadas equivalen a entre 10 y unos 180 resultados independientes (calculado por el revisor).
2	**Los filtros de `_study.yaml` :** firmarlos como están, darlos tú, o pedir una calibración nueva con los spreads buenos.	Dejar de tener cargadas cifras de un estudio borrado.	Una calibración nueva gasta CPU y, si mira `oos1`, lo gasta en los activos que toque. Firmarlos tal cual no cuesta nada porque hoy casi no deciden (decisión 3).
3	**El tope de minutos y el databank lleno:** los 90 minutos ya los dejaste como están (2026-10-07); queda abierta la pregunta de los 180 de `Trade_` / `Research_` con el databank lleno, o acortar hasta que no se llene.	Saber si los minutos de más dan supervivientes o sólo ajuste al `build`.	La prueba es una celda construida dos veces (corto y largo) con réplicas; el genético varía mucho de una corrida a otra, así que una pareja no basta.
4	**La métrica del genético:** dejar Ret/DD, o fijarla tú y escribirla en `_build.yaml`.	Que deje de depender del donante.	Escribirla es un cambio pequeño. Compararla con otras son varias réplicas por métrica.
5	**El suelo de 35 operaciones al año:** dentro del build o sólo en el paso 8.	Dentro: el databank no se llena de estrategias por debajo de tu suelo de 35 (6.088 de 10.000 en la corrida); el paso 8 tira sólo las de menos de 20, unas 3.900, y deja las demás en Limbo (`criteria.yaml:79-80`).	No se sabe qué entraría en su lugar. La evidencia de que dentro salía peor es de la calibración borrada.
6	**La salida por condición con plantilla:** aceptar que no existe, o pedir un hueco de salida en las plantillas.	Con hueco, la doctrina «optativa» sería verdad y habría estrategias que salen por regla.	Más formas de regla posibles (hasta 1 + 103 + 5.253 por la salida con una paleta de 103). Los ensayos serían los mismos, porque el build para por tiempo: se repartirían entre más formas.
7	**El `MonteCarloManipulation` del build:** apagarlo o darle condiciones.	Apagado: CPU. Con condiciones: un filtro más dentro del build.	Un filtro dentro vuelve a ser una selección sobre el `build` que habría que medir.
8	**La banda 90-170 de la paleta:** que el aplicador se niegue fuera de ella, o que siga avisando.	Que no se pueda construir con una paleta de 6 condiciones por descuido.	Las paletas viejas fuera de banda dejarían de poder aplicarse.

6.4 El límite que no cambia

El build mira un solo tramo y lo mira cientos de miles de veces. Lo que sale de él es una lista de candidatas ordenada por lo bien que se ajustaron a ese tramo. Ningún ajuste de este paso convierte esa lista en evidencia: eso empieza en el paso 8, sobre `oos1`, y lo que el paso 6 puede aportar es decir con claridad cuántas se probaron para llegar hasta ahí.